

Ex.no:5	DATA VISUALIZATION USING MATPLOTLIB – SALES DATA
Date:	

Aim:

To visualize dataset features and distributions using line charts, bar charts, pie charts, histograms, scatter plots, and box plots using Matplotlib.

Algorithm:**ALGORITHM 1: LINE CHART**

Objective: To visualize the monthly sales trend.

Steps

1. Load the sales dataset.
2. Select the Month and Sales columns.
3. Plot the monthly sales using a line chart.
4. Add title and axis labels.
5. Display the graph.

Interpretation

The line chart shows how sales vary across different months and helps identify the sales trend.

Program 1: Line Chart

```
months = ["January", "February", "March", "April", "May", "June"]
```

```
sales = [100, 150, 120, 180, 200, 250]
```

```
plt.plot(months, sales)
```

```
plt.title("Monthly Sales")
```

```
plt.xlabel("Months")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

ALGORITHM 2: BAR CHART

Objective: To compare sales across different months.

Steps

1. Select the Month and Sales columns.
2. Create a bar chart using the monthly sales values.
3. Add title and axis labels.
4. Display the chart.

Interpretation

The bar chart helps compare the sales values across different months.

```
# Program 2: Bar Chart
```

```
plt.bar(months, sales)
```

```
plt.title("Monthly Sales")
```

```
plt.xlabel("Months")
```

```
plt.ylabel("Sales")
```

```
plt.show()
```

ALGORITHM 3: PIE CHART

Objective: To visualize the distribution of sales across different months.

Steps

1. Select the sales values.
2. Create a pie chart using the sales values.
3. Display percentage labels.
4. Add a suitable title.
5. Display the chart.

Interpretation

The pie chart shows the percentage contribution of each month to the total sales.

```
# Program 3: Pie Chart
```

```
plt.pie(sales, labels=months, autopct="%1.1f%%")
```

```
plt.title("Sales Distribution")
```

```
plt.show()
```

ALGORITHM 4: HISTOGRAM

Objective: To visualize the distribution of sales values.

Steps

1. Select the Sales column.
2. Divide the sales values into bins.
3. Create a histogram.
4. Add title and axis labels.
5. Display the histogram.

Interpretation

The histogram shows the frequency distribution of the monthly sales values.

```
# Program 4: Histogram
```

```
plt.hist(sales, bins=5)
```

```
plt.title("Sales Distribution")
```

```
plt.xlabel("Sales")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

ALGORITHM 5: SCATTER PLOT

Objective: To visualize the sales values for different months.

Steps

1. Select Month as the X-axis variable.
2. Select Sales as the Y-axis variable.
3. Create a scatter plot.
4. Add title and axis labels.
5. Display the graph.

Interpretation

The scatter plot shows the variation in sales across different months.

```
# Program 5: Scatter Plot
```

```
plt.scatter(months, sales)
```

```
plt.title("Monthly Sales")
```

```
plt.xlabel("Months")
```

```
plt.ylabel("Sales")  
plt.show()
```

ALGORITHM 6: BOX PLOT

Objective: To visualize the distribution of monthly sales.

Steps

1. Select the Sales column.
2. Create a box plot.
3. Add a suitable title and axis label.
4. Display the plot.

Interpretation

The box plot shows the distribution of sales values and helps identify possible outliers.

Program 6: Box Plot

```
plt.boxplot(sales)  
plt.title("Sales Box Plot")  
plt.ylabel("Sales")  
plt.show()
```

ALGORITHM 7: MULTIPLE LINE CHARTS

Objective: To compare sales and profit across different months.

Steps

1. Store the profit values.
2. Plot the sales values using a line chart.
3. Plot the profit values using another line chart.
4. Add title, axis labels and legend.
5. Display the graph.

Interpretation

The multiple line chart compares the sales and profit values across different months.

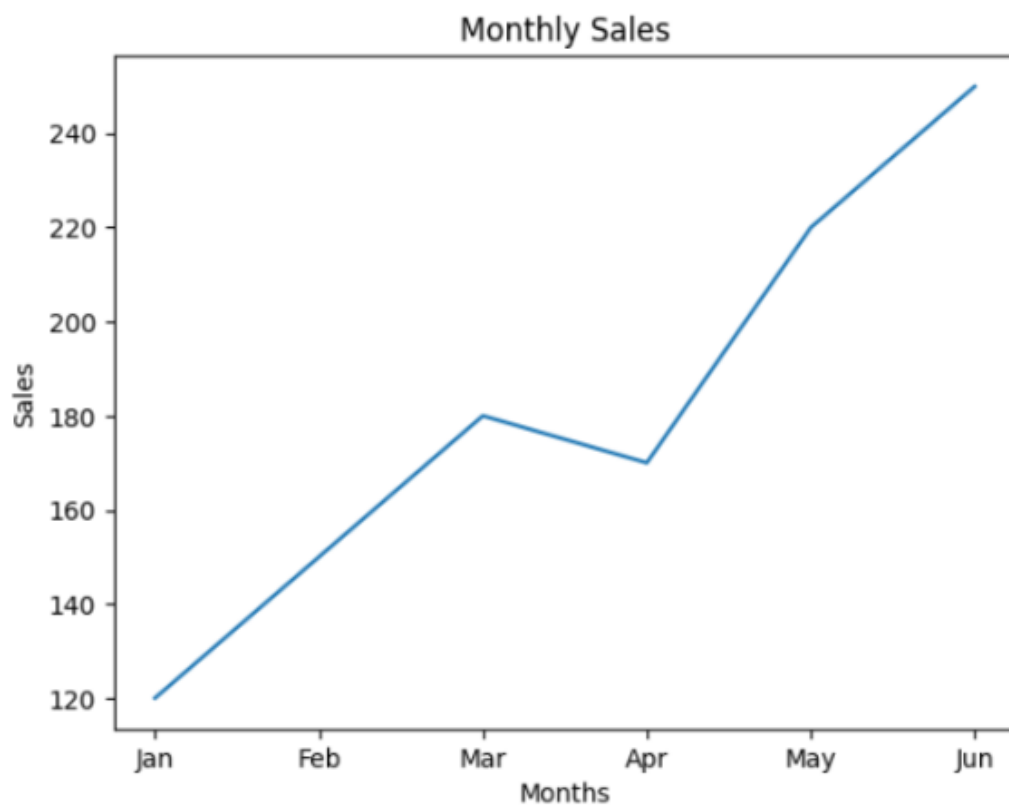
Program 7: Multiple Line Charts

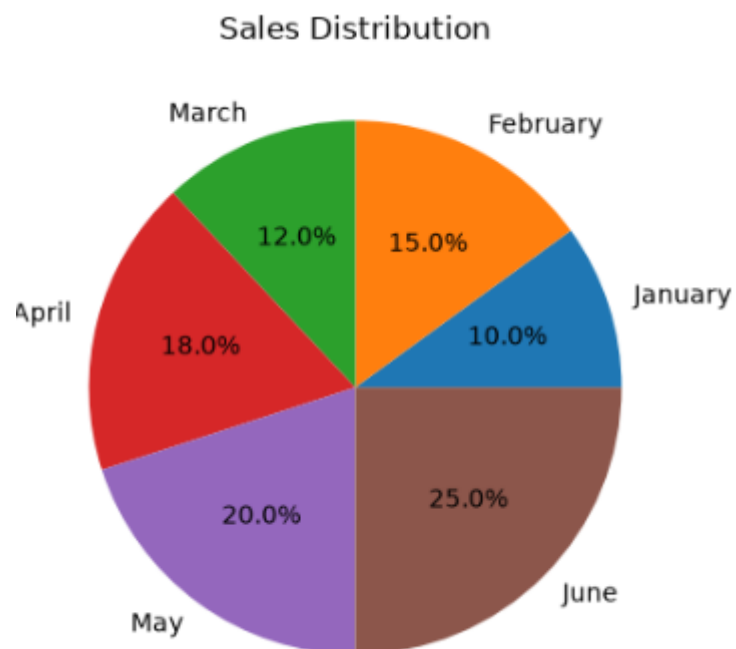
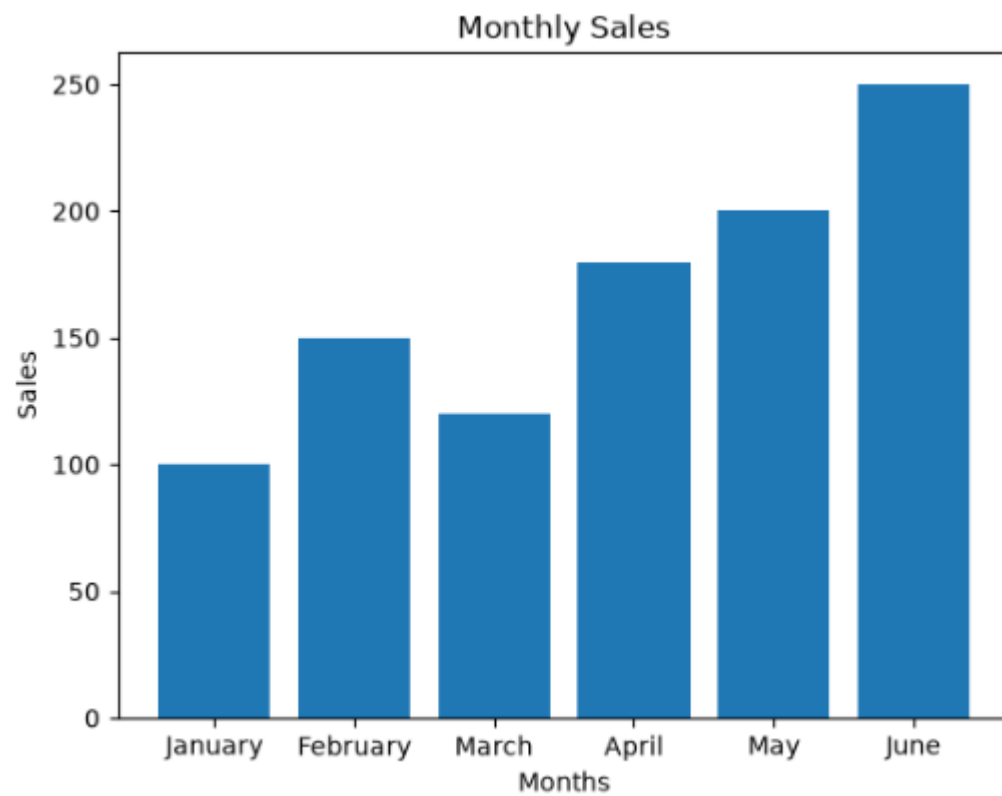
```
profit = [20, 30, 40, 35, 50, 60]
```

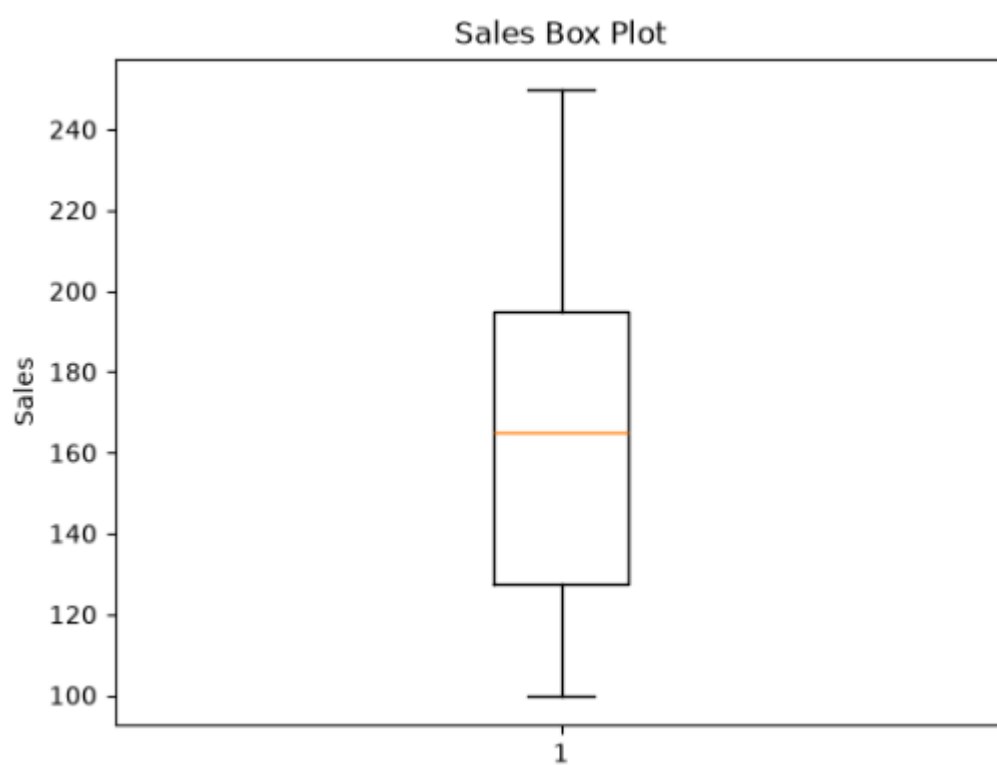
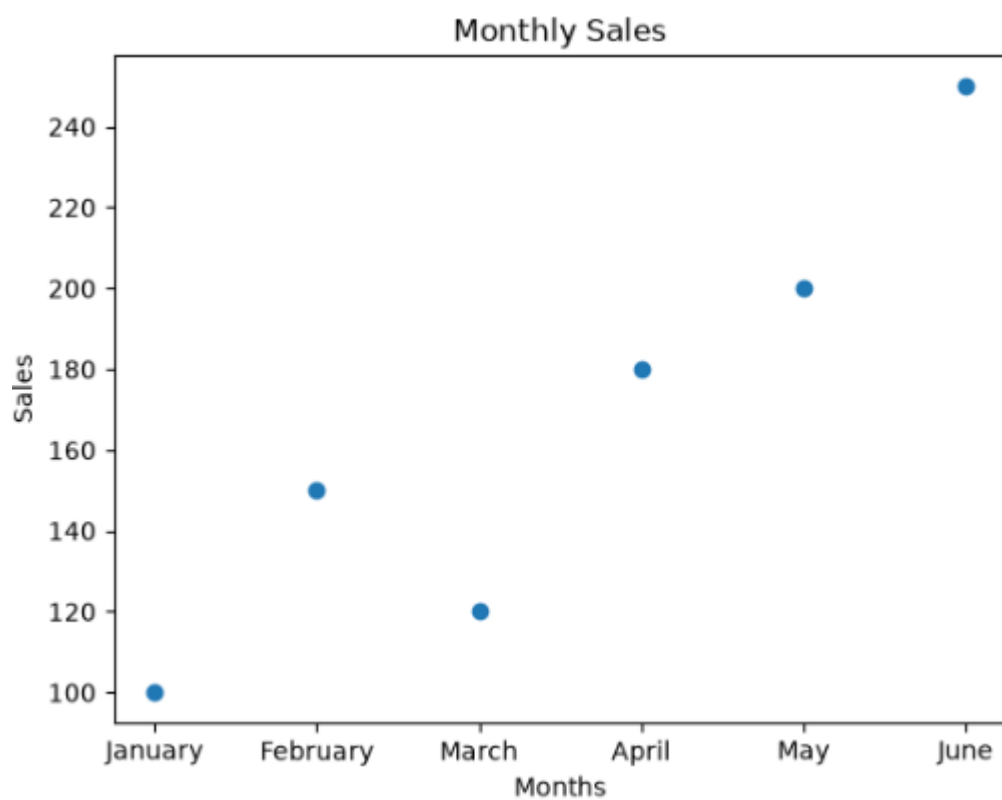
```
plt.plot(months, sales, label="Sales")
```

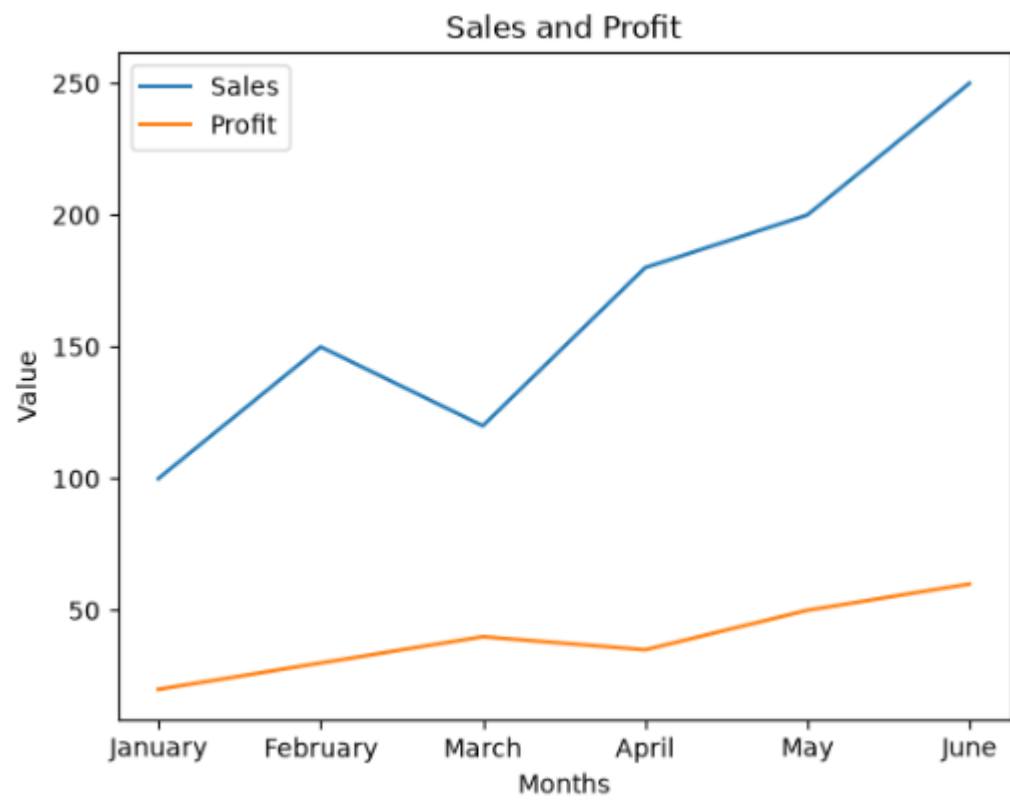
```
plt.plot(months, profit, label="Profit")  
plt.title("Sales and Profit")  
plt.xlabel("Months")  
plt.ylabel("Value")  
plt.legend()  
plt.show()
```

Output:









Result: